

République du Sénégal



MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE

DIRECTION GÉNÉRALE DE L'ENSEIGNEMENT SUPÉRIEUR



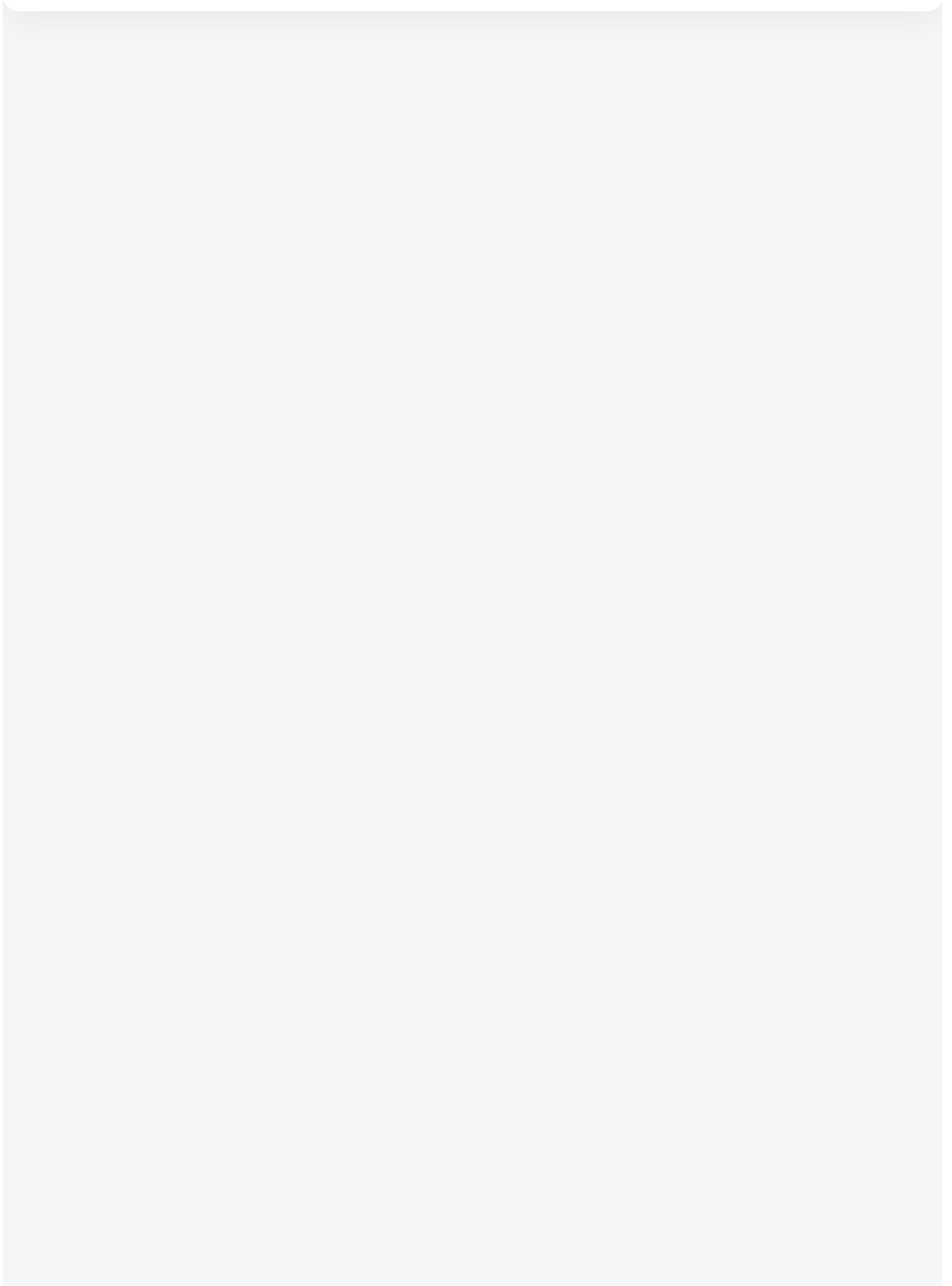
Rapport DevOps — AWS CodeCommit

Présenté par : **Ibrahima Khalilou Thiam**

Master 2 Génie Logiciel UDB

Matricule 003

À **M. Massamba LO**



Sommaire

1. [Objectifs et contexte](#)
2. [Prérequis](#)
3. [Concepts et architecture](#)
4. [Installation et configuration](#)
5. [Réalisation pratique](#)
6. [Commandes et explications](#)
7. [Vérifications et tests](#)
8. [Dépannage](#)
9. [Sécurité et bonnes pratiques](#)
10. [Checklist](#)
11. [Conclusion](#)

1. Objectifs et contexte

AWS CodeCommit est un service Git managé permettant d'héberger des dépôts privés dans AWS. Dans une démarche DevOps, il constitue la couche de gestion du code source et peut alimenter des chaînes CI/CD avec CodePipeline et CodeBuild.

- Créer et administrer un dépôt Git CodeCommit.
- Configurer une authentification Git sécurisée.
- Cloner, modifier, commiter et pousser du code.
- Organiser le développement avec des branches.
- Intégrer le dépôt à une chaîne CI/CD.

Point d'architecture : CodeCommit stocke le code ; IAM contrôle les autorisations ; CodePipeline orchestre les étapes ; CodeBuild compile et teste ; CodeDeploy peut assurer le déploiement.

2. Prérequis

- Un compte AWS et une région AWS choisie, par exemple `us-east-1`.
- AWS CLI installée et authentifiée.
- Git installé sur le poste de travail.
- Un principal IAM disposant uniquement des permissions nécessaires.
- Pour une chaîne CI/CD : CodePipeline, CodeBuild et éventuellement CodeDeploy.

```
aws --version git --version aws sts get-caller-identity
```

Remarque : les commandes de ce rapport utilisent `us-east-1` comme exemple. Adapter la région à l'environnement réellement utilisé.

3. Concepts et architecture

Le flux de travail repose sur Git et les services AWS :

1. Le développeur travaille dans un dépôt local Git.
2. `git push` envoie un commit vers CodeCommit.
3. CodePipeline détecte le changement via l'intégration événementielle.
4. CodeBuild récupère le code et exécute compilation et tests.
5. Les artefacts validés sont transmis à l'étape de déploiement.

Développeur → Git local → CodeCommit → CodePipeline → CodeBuild → Déploiement

Déclenchement CI/CD : pour CodePipeline, l'approche moderne privilégie les événements Amazon EventBridge plutôt qu'un polling permanent. Si EventBridge est utilisé, le polling de la source doit être désactivé dans la configuration du pipeline.

4. Installation et configuration

4.1 Vérifier l'identité AWS

```
aws sts get-caller-identity
```

4.2 Configurer le credential helper HTTPS

```
git config --global credential.helper '!aws codecommit credential-helper $@' git  
config --global credential.UseHttpPath true git config --global --get-regexp  
credential
```

Le credential helper permet à Git d'obtenir les informations d'authentification à partir des credentials AWS disponibles dans l'environnement. Il évite d'inscrire un secret AWS directement dans l'URL du dépôt.

Attention : ne placez jamais une clé d'accès AWS, un secret ou un mot de passe dans le dépôt Git, un script versionné ou une URL de clone.

5. Réalisation pratique

5.1 Créer le dépôt

```
aws codecommit create-repository \ --repository-name mon-projet-dev \ --repository-  
description "Projet Master 2 - Application web" \ --region us-east-1
```

5.2 Vérifier le dépôt

```
aws codecommit get-repository \ --repository-name mon-projet-dev \ --region us-east-1
```

5.3 Cloner le dépôt

```
git clone https://git-codecommit.us-east-1.amazonaws.com/v1/repos/mon-projet-dev cd  
mon-projet-dev
```

 Clonage d'un dépôt CodeCommit

5.4 Premier commit

```
echo "# Mon Projet Dev" > README.md git add README.md git commit -m "Initialisation  
du projet" git branch -M main git push -u origin main
```

5.5 Créer une branche fonctionnelle

```
git switch -c feature/ajout-login echo "Page de connexion" > login.html git add  
login.html git commit -m "Ajout de la page de connexion" git push -u origin  
feature/ajout-login
```

6. Commandes et explications

Commande	Rôle
<code>aws sts get-caller-identity</code>	Identifie le principal AWS utilisé.
<code>aws codecommit create-repository</code>	Crée un dépôt.
<code>aws codecommit get-repository</code>	Retourne les métadonnées du dépôt.
<code>git clone</code>	Crée une copie locale du dépôt.
<code>git status</code>	Affiche l'état du répertoire de travail.
<code>git add</code>	Place les modifications dans l'index.
<code>git commit</code>	Crée un commit local.
<code>git push</code>	Publie les commits vers CodeCommit.
<code>git pull</code>	Récupère et intègre les changements distants.
<code>git fetch</code>	Récupère les références distantes sans fusion automatique.

Branches et fusion

`git branch` `git switch main` `git merge feature/ajout-login` `git push origin main`

Organisation : une stratégie `feature/*` → revue/validation → `main` est préférable à des modifications directes non contrôlées sur la branche principale.

7. Vérifications et tests

7.1 Vérifier le dépôt AWS

```
aws codecommit get-repository --repository-name mon-projet-dev --region us-east-1
```

7.2 Vérifier le dépôt Git local

```
git remote -v  
git status  
git branch -a  
git log --oneline -5
```

7.3 Tester le cycle push/pull

```
echo "Test CodeCommit" > test.txt  
git add test.txt  
git commit -m "Test du dépôt distant"  
git push origin main  
git pull origin main
```

7.4 Vérifier les branches distantes

```
git ls-remote --heads origin
```

Critère de réussite : le commit doit être visible dans CodeCommit, la branche distante doit être accessible et aucun secret ne doit apparaître dans l'historique Git.

8. Dépannage

Problème	Diagnostic / correction
403 / accès refusé	Exécuter <code>aws sts get-caller-identity</code> puis vérifier les permissions IAM et la région.
Repository not found	Vérifier le nom du dépôt, la région et l'URL <code>origin</code> .
Credential helper invalide	Vérifier <code>git config --global --get-regexp credential</code> et la présence d'AWS CLI.
Push rejeté	Exécuter <code>git fetch</code> , examiner les divergences puis fusionner ou rebaser selon la politique de l'équipe.
Conflit Git	Résoudre les fichiers en conflit, exécuter <code>git add</code> , puis <code>git commit</code> et <code>git push</code> .
Pipeline non déclenché	Vérifier l'intégration CodeCommit/EventBridge/CodePipeline et éviter un polling simultané inutile.

9. Sécurité et bonnes pratiques

- Ne pas utiliser le compte AWS root pour Git au quotidien.
- Appliquer le principe du moindre privilège avec IAM.
- Préférer des identités temporaires ou des rôles lorsque l'architecture le permet.
- Ne jamais versionner de secrets, clés privées, mots de passe ou fichiers de credentials.
- Protéger la branche principale par une procédure de revue et de validation.
- Journaliser et surveiller les opérations AWS avec les services adaptés.
- Utiliser HTTPS avec le credential helper ou SSH correctement configuré.
- Pour CI/CD, utiliser des rôles IAM dédiés aux services plutôt que des clés statiques.

Principe essentiel : les droits IAM doivent être limités au dépôt et aux actions réellement nécessaires. Éviter d'attribuer systématiquement des politiques administratives ou `FullAccess`.

10. Checklist

- ☐ AWS CLI installée et fonctionnelle
- ☐ `aws sts get-caller-identity` retourne une identité valide
- ☐ Dépôt CodeCommit créé dans la bonne région
- ☐ Credential helper Git configuré
- ☐ Dépôt cloné avec succès
- ☐ Commit et push vers `main` réalisés
- ☐ Branche fonctionnelle créée et poussée
- ☐ Cycle fetch/pull testé
- ☐ Permissions IAM limitées au besoin
- ☐ Aucun secret présent dans le dépôt
- ☐ Intégration CI/CD vérifiée si utilisée

11. Conclusion

Cette étude pratique a permis de mettre en place AWS CodeCommit comme composant de gestion du code source dans une architecture DevOps. La configuration du credential helper, les opérations Git, la gestion des branches et les contrôles de sécurité constituent les bases nécessaires à une utilisation professionnelle.

Associé à CodePipeline, CodeBuild et CodeDeploy, CodeCommit peut s'intégrer dans une chaîne CI/CD automatisée. La qualité de cette architecture dépend toutefois de la maîtrise des permissions IAM, de la gestion des secrets et des mécanismes de déclenchement des pipelines.